

Análise Crítica de Artigo Científico sobre Testes de Software

Antonio Matheus Pereira de Lima¹, Fernanda Vitória Araújo Santos²,
João Guilherme da Silva³, Juan do Monte Pereira⁴

¹Análise em Desenvolvimento de Sistemas – Instituto Federal do Piauí (IFPI)
Caixa Postal gabinete.capic@ifpi.edu.br – 64605-5

Resumo. *Este trabalho apresenta uma análise crítica do artigo científico Automated NFR testing in continuous integration environments: a multi-case study of Nordic companies, publicado no periódico Empirical Software Engineering em 2023. O estudo original investiga, por meio de um estudo de caso múltiplo realizado em quatro empresas nórdicas, como testes automatizados de Requisitos Não Funcionais (NFRs) podem ser habilitados e apoiados por ambientes de Integração Contínua (CI). A presente análise discute o problema abordado, a metodologia empregada, os principais resultados obtidos, bem como as limitações do estudo e sua aplicabilidade prática em empresas de desenvolvimento de software.*

1. Introdução

O presente trabalho consiste na análise crítica do artigo científico intitulado *Automated NFR testing in continuous integration environments: a multi-case study of Nordic companies*. Nesta seção, são apresentados os principais dados de identificação da publicação.

- **Título do Artigo:** *Automated NFR testing in continuous integration environments: a multi-case study of Nordic companies.*
- **Autores:**
 1. Liang Yu;
 2. Emil Alégroth;
 3. Panagioti Chatzipetrou;
 4. Tony Gorschek.
- **Instituição dos Autores:**
 - Blekinge Institute of Technology, Karlskrona, Suécia;
 - Örebro University School of Business (CERIS), Örebro, Suécia;
 - Fortiss GmbH, Munique, Alemanha.
- **Ano de Publicação:**
 - Aceito em: 7 de junho de 2023;
 - Publicado online em: 24 de outubro de 2023.
- **Periódico:**
 - *Empirical Software Engineering*, volume 28, artigo 144.
- **Área de Pesquisa:**
 - Engenharia de Software, com foco em testes automatizados de requisitos não funcionais (*Non-Functional Requirements – NFRs*) em ambientes de Integração Contínua (*Continuous Integration – CI*).

2. Contextualização

2.1. Problema abordado

O artigo investiga como habilitar e apoiar a realização de testes automatizados de Requisitos Não Funcionais (*Non-Functional Requirements – NFRs*) em ambientes de Integração Contínua (*Continuous Integration – CI*). Embora a automação de testes funcionais esteja amplamente consolidada, a automação de testes de NFRs ainda representa um desafio significativo para a indústria de software.

2.2. Importância do problema

Os requisitos não funcionais, como desempenho, segurança e confiabilidade, influenciam diretamente a qualidade e o sucesso de um produto de software. Além disso, em diversos contextos, esses requisitos precisam atender a regulamentações específicas, como o Regulamento Geral sobre a Proteção de Dados (*General Data Protection Regulation – GDPR*), tornando sua validação essencial durante o desenvolvimento.

2.3. Impactos no desenvolvimento de software

A ausência de processos automatizados para testes de NFRs impacta negativamente o desenvolvimento de software, pois muitas organizações ainda dependem de práticas predominantemente manuais. Isso torna o processo de validação menos eficiente, cria gargalos nos ambientes de CI e aumenta a probabilidade de que problemas sejam identificados apenas nas fases finais do desenvolvimento. Segundo o artigo, defeitos relacionados a requisitos não funcionais podem gerar retrabalho equivalente a 40% a 50% do esforço total de um projeto, elevando custos e comprometendo a qualidade do software entregue.

3. Fundamentação

Para compreender o estudo analisado, é necessário conhecer alguns conceitos fundamentais relacionados à Engenharia de Software.

3.1. Testes Automatizados

Os testes automatizados consistem na utilização de ferramentas e scripts para executar casos de teste de forma automática, reduzindo a necessidade de execução manual. Essa abordagem torna o processo de validação mais rápido, repetitivo e confiável, além de facilitar a identificação precoce de defeitos durante o desenvolvimento do software.

3.2. Integração Contínua

A Integração Contínua (*Continuous Integration – CI*) é uma prática de desenvolvimento de software na qual alterações no código são integradas frequentemente a um repositório compartilhado. A cada integração, processos automatizados, como compilação e execução de testes, são realizados para verificar a qualidade do software e detectar falhas rapidamente.

3.3. Requisitos Não Funcionais

Os Requisitos Não Funcionais (*Non-Functional Requirements – NFRs*) descrevem características relacionadas à qualidade do sistema, definindo como ele deve executar suas funcionalidades. Entre os principais exemplos estão desempenho, segurança, confiabilidade, disponibilidade, escalabilidade e usabilidade.

4. Objetivo da Pesquisa

4.1. Objetivo principal

O objetivo principal do artigo é examinar como o teste automatizado de Requisitos Não Funcionais (*Non-Functional Requirements – NFRs*) pode ser habilitado e suportado por ambientes de Integração Contínua (*Continuous Integration – CI*) em empresas de desenvolvimento de software.

4.2. Contribuição esperada

Os autores buscam demonstrar como os dados produzidos pelos ambientes de CI podem ser utilizados para avaliar e monitorar continuamente a qualidade dos requisitos não funcionais implementados, contribuindo para a identificação precoce de problemas e para a melhoria contínua da qualidade do software.

4.3. Investigação realizada

Para alcançar o objetivo proposto, os autores conduziram a investigação por meio de quatro perguntas de pesquisa, que orientaram a coleta e a análise dos dados:

- **RQ1:** Quais tipos de NFRs são verificados por meio de testes automatizados?
- **RQ2:** Quais métricas são utilizadas para testes automatizados de NFRs na prática industrial?
- **RQ3:** Como os ambientes de Integração Contínua contribuem para as capacidades de teste de NFRs?
- **RQ4:** Quais desafios estão associados aos testes automatizados de NFRs?

As perguntas de pesquisa delimitam os aspectos específicos analisados: os tipos de NFRs testados (RQ1), as métricas empregadas (RQ2), a contribuição dos componentes de CI (RQ3) e os desafios enfrentados na prática (RQ4).

5. Metodologia

5.1. Tipo de pesquisa

A pesquisa foi conduzida por meio de um estudo de caso múltiplo, com o objetivo de investigar como empresas de desenvolvimento de software utilizam ambientes de Integração Contínua para apoiar a automação de testes de Requisitos Não Funcionais.

5.2. Etapas da pesquisa

O processo metodológico foi dividido em quatro etapas:

1. Planejamento, com definição dos objetivos e das questões de pesquisa;
2. Elaboração do desenho do estudo de caso, incluindo roteiro das entrevistas, seleção dos participantes e testes-piloto;
3. Coleta de dados por meio de entrevistas semiestruturadas;
4. Análise temática e síntese dos dados coletados.

5.3. Coleta e análise dos dados

A coleta de dados foi realizada por meio de 22 entrevistas semiestruturadas com profissionais da indústria de software. Após a transcrição das entrevistas, os dados foram analisados utilizando a técnica de análise temática.

5.4. Estudo de caso

O estudo foi realizado em quatro empresas de desenvolvimento de software: Qvantel Sweden, Ericsson, Fortnox e uma empresa identificada como Company Alpha, cujo nome foi preservado por questões de confidencialidade. Ao todo, foram analisados cinco projetos distintos, identificados pelos autores como Projetos A, B, C, D e E.

6. Solução Proposta

6.1. Funcionamento da solução

A solução proposta consiste em um modelo teórico-empírico que demonstra como os ambientes de Integração Contínua podem habilitar e dar suporte à automação dos testes de Requisitos Não Funcionais. O modelo mostra como os dados produzidos durante o processo de desenvolvimento são transformados em métricas capazes de avaliar continuamente a qualidade desses requisitos.

6.2. Etapas do modelo

As etapas do modelo são apresentadas na Figura 1.

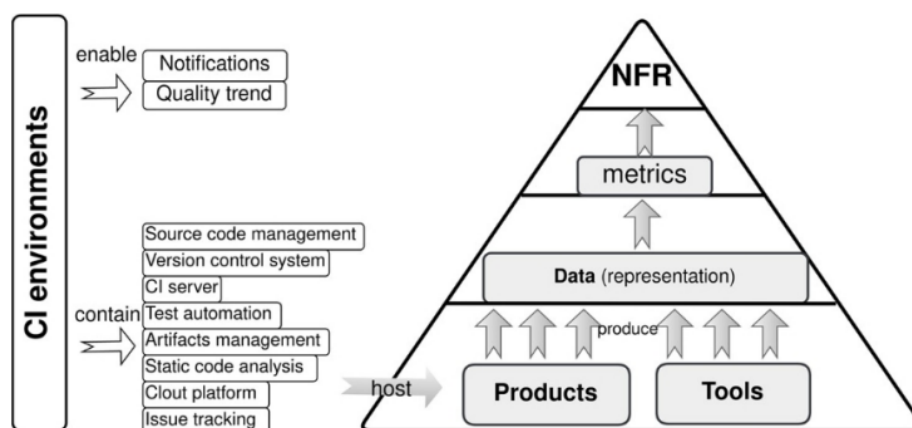


Figura 1. Figura 4 do artigo - Modelo de suporte aos testes automatizados de requisitos não funcionais em ambientes de Integração Contínua.

Fonte: Yu et al. (2023).

Conforme ilustrado na Figura 1, os componentes do ambiente de Integração Contínua, como gerenciamento de código-fonte, controle de versão, servidor de CI, automação de testes, gerenciamento de artefatos, análise estática de código, plataforma em nuvem e rastreamento de problemas, produzem dados juntamente com os produtos desenvolvidos e as ferramentas utilizadas. Esses dados são processados e convertidos em métricas específicas, que permitem avaliar continuamente os requisitos não funcionais do software, além de gerar notificações e acompanhar a evolução da qualidade ao longo do tempo.

6.3. Entradas

O modelo recebe como entrada informações produzidas durante o processo de desenvolvimento, como alterações no código-fonte, *commits* realizados no sistema de controle de versão e artefatos gerados durante o processo de *build*.

6.4. Saídas

Como resultado do processamento dessas informações, o ambiente de Integração Contínua produz métricas de qualidade, relatórios de vulnerabilidades, indicadores de desempenho, tendências de qualidade ao longo do tempo e notificações automáticas que auxiliam os desenvolvedores na identificação e correção de problemas relacionados aos requisitos não funcionais.

7. Resultados

7.1. Principais resultados

Os resultados da pesquisa mostram que os ambientes de Integração Contínua possibilitam a automação de diferentes tipos de requisitos não funcionais. A Figura 2 apresenta uma síntese dos dados coletados durante o estudo, relacionando os grupos de código analisados, os tipos de NFRs identificados, as ferramentas utilizadas, os componentes de CI empregados e os projetos em que cada requisito foi observado.

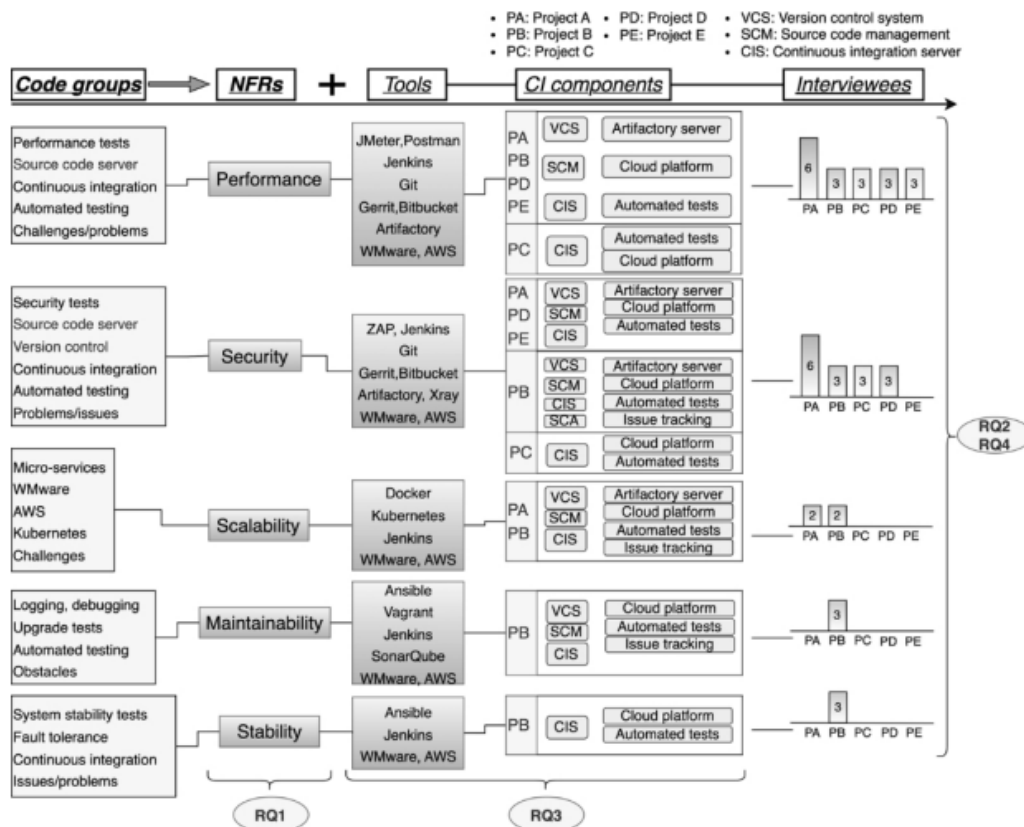


Fig. 3 Síntese dos dados coletados, incluindo tipos de NFRs, ferramentas e componentes de infraestrutura de código (CI)

Figura 2. Figura 3 do artigo - Síntese dos dados coletados, incluindo tipos de NFRs, ferramentas e componentes de infraestrutura de código (CI).

Fonte: Yu et al. (2023).

Conforme ilustrado na Figura 2, foram identificados cinco tipos de requisitos não funcionais: desempenho, segurança, escalabilidade, manutenibilidade e estabilidade. O

requisito de desempenho foi observado em todos os projetos analisados, enquanto os requisitos de segurança estiveram presentes em quatro projetos. Já os requisitos de escalabilidade, manutenibilidade e estabilidade apareceram em menor frequência. A figura também evidencia a associação entre cada tipo de NFR, as ferramentas utilizadas para sua avaliação e os componentes de Integração Contínua empregados pelas empresas participantes.

7.2. Validação da proposta

Os resultados obtidos indicam que a proposta apresentada pelos autores mostrou-se viável para apoiar a automação de testes de requisitos não funcionais. O estudo resultou em um modelo teórico-empírico e em um guia prático que relaciona componentes dos ambientes de Integração Contínua, ferramentas e métricas utilizadas na avaliação contínua dos NFRs em empresas de desenvolvimento de software.

7.3. Métricas utilizadas

As métricas utilizadas para avaliar os requisitos não funcionais variaram de acordo com o tipo de NFR analisado. A Figura 3 apresenta exemplos das métricas extraídas nos projetos estudados, bem como os componentes de Integração Contínua relacionados à sua obtenção.

→ Tabela 6 Métricas de NFR extraídas para testes automatizados de NFR nos projetos industriais estudados

Tipo de NFR	Métricas de NFR	Dados nos projetos estudados	Componentes de CI relacionados	Nome do projeto
Desempenho	Tempo médio de resposta (MRT): $MRT = (A1 + A2 + \dots + An)/n$, onde Ai é o tempo que um servidor leva para responder a uma solicitação, e n é o número de respostas medidas.	Tempo de resposta de transações de pagamento entre contas em um serviço financeiro	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto A
		Tempo de resposta do gerenciamento de assinaturas de usuários em um serviço de rede móvel	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto B
		Tempo de resposta de solicitações para criar usuários e contas em um sistema bancário	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto C
	Utilização do processador (PU): $PU = A/B$, em que A é o tempo de processador necessário para executar um conjunto de tarefas e B é o tempo de operação necessário para realizar as tarefas. Precisão das solicitações de API (AOAR): $AOAR = A/B$, em que A é o número de solicitações de API com falha e B é o número total de solicitações de API processadas por um serviço.	Tempo de processamento para gerenciar dispositivos móveis e dados em um serviço web	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes Plataforma em nuvem	Projeto D
		Taxa de precisão das solicitações de usuários em um sistema de assistência social	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto E
Segurança	Criptografia de dados (DEC): $DEC = A/B$, em que A é o número de itens de dados do usuário criptografados corretamente e B é o número total de itens de dados que requerem criptografia.	Dados de privacidade do usuário em um sistema financeiro	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes Rastreamento de problemas	Projeto A

Figura 3. Tabela 6 do artigo - Métricas de requisitos não funcionais extraídas para testes automatizados nos projetos estudados.

Fonte: Yu et al. (2023).

Conforme apresentado na Figura 3, para os requisitos de desempenho foram utilizadas métricas como o Tempo Médio de Resposta (*Mean Response Time – MRT*), a Utilização do Processador (*Processor Usage – PU*) e a Precisão das Solicitações de API

(*Accuracy of API requests – AOAR*). Para os requisitos de segurança, foram empregadas métricas relacionadas à criptografia de dados e à avaliação de vulnerabilidades. Essas métricas foram obtidas a partir de dados coletados pelos componentes do ambiente de Integração Contínua, como sistemas de controle de versão, servidores de CI, ferramentas de automação de testes, plataformas em nuvem e sistemas de rastreamento de problemas.

7.4. Comparação com abordagens tradicionais

Os autores observam que a utilização de ambientes de Integração Contínua proporciona vantagens em relação aos processos tradicionais de testes predominantemente manuais. Entre os principais benefícios estão a detecção precoce de problemas, o monitoramento contínuo da qualidade dos requisitos não funcionais e a geração de feedback automático aos desenvolvedores, tornando o processo de desenvolvimento mais eficiente.

8. Limitações

8.1. Aspectos não resolvidos

Os autores destacam que alguns requisitos não funcionais, especialmente os de natureza subjetiva, como a usabilidade, ainda são difíceis de automatizar e medir de forma confiável. Dessa forma, esses requisitos continuam dependendo, em grande parte, de avaliações realizadas por especialistas ou usuários.

8.2. Dificuldades encontradas

Durante o estudo, foram identificados os seguintes desafios:

- **Interdependência entre submódulos do sistema:** alterações realizadas para atender a um requisito não funcional em um módulo podem causar falhas inesperadas em outros módulos devido às dependências existentes.
- **Dificuldade na análise de causa-raiz:** como os requisitos não funcionais são distribuídos entre diferentes equipes, identificar a origem de um problema torna-se um processo complexo e demorado.
- **Rastreamento do ciclo de vida dos NFRs:** há pouca visibilidade para acompanhar um requisito não funcional desde sua implementação até a versão final do software, e o rastreamento manual pode gerar inconsistências.
- **Instabilidade dos ambientes de Integração Contínua:** limitações de hardware ou problemas de rede podem provocar falsos positivos nos testes, afetando principalmente requisitos relacionados ao desempenho, como tempo de resposta e latência.
- **Análise dos relatórios de testes:** embora os dados sejam gerados, muitas empresas não os armazenam de forma centralizada devido à falta de conhecimento sobre quais métricas monitorar, dificultando a análise dos resultados.

8.3. Ameaças à validade

Os autores discutem quatro ameaças à validade do estudo:

- **Validade interna:** possibilidade de interpretação incorreta das perguntas das entrevistas, mitigada por meio de testes-piloto.
- **Validade externa:** o estudo foi realizado apenas em empresas dos países nórdicos, limitando a generalização dos resultados para outros contextos.

- **Validade de construto:** alguns participantes poderiam deixar de relatar desafios reais, risco reduzido pela garantia de anonimato.
- **Validade de conclusão:** a análise temática dos dados envolve certo grau de subjetividade na interpretação dos resultados.

8.4. Melhorias propostas

Como trabalhos futuros, os autores sugerem:

- aprofundar a investigação sobre o papel dos componentes dos ambientes de Integração Contínua no suporte aos testes de requisitos não funcionais;
- implementar sondas baseadas em algoritmos de aprendizado de máquina para analisar os dados produzidos pelos testes de NFRs;
- aperfeiçoar os ambientes de Integração Contínua para viabilizar testes orientados por dados (*data-driven testing*).

9. Aplicação Prática

9.1. Tipo de empresa

A abordagem apresentada no artigo pode ser aplicada em empresas de desenvolvimento de software que utilizam ambientes de Integração Contínua (CI) em seus processos. Ela é especialmente útil para organizações que precisam monitorar continuamente requisitos não funcionais, como desempenho, segurança, escalabilidade e manutenibilidade.

9.2. Tipos de projetos

Pode ser empregada em aplicações web, sistemas corporativos, serviços em nuvem e outros projetos em que a qualidade dos requisitos não funcionais seja um fator importante.

9.3. Etapa de aplicação

A abordagem é aplicada durante todo o processo de desenvolvimento, sendo integrada ao ambiente de Integração Contínua. Dessa forma, sempre que novas alterações são incorporadas ao código, os testes automatizados podem ser executados e as métricas dos requisitos não funcionais atualizadas continuamente.

9.4. Ferramentas utilizadas

As ferramentas dependem do tipo de requisito não funcional que será avaliado. No estudo, foram utilizadas ferramentas como JMeter, JUnit, Postman, ZAP, Anchore e SonarQube para a execução dos testes, além de Jenkins, Bitbucket, VMware e JFrog Artifactory para apoiar o processo de Integração Contínua e o gerenciamento das atividades.

9.5. Avaliação da solução

Com base nos resultados apresentados no artigo, considera-se que a solução possui potencial para melhorar a automação dos testes de requisitos não funcionais em organizações que já utilizam Integração Contínua. A abordagem permite o monitoramento contínuo da qualidade do software e a detecção precoce de problemas.

10. Análise Crítica

10.1. Qualidade da redação

O artigo é bem estruturado e apresenta uma organização clara das informações. A divisão em introdução, fundamentação teórica, metodologia, resultados, discussão e conclusão facilita a compreensão do estudo. Além disso, os autores utilizam tabelas e figuras que contribuem para a interpretação dos resultados e para a apresentação do modelo proposto.

10.2. Avaliação da metodologia

A metodologia adotada mostrou-se adequada aos objetivos da pesquisa. O estudo de caso múltiplo permitiu analisar práticas utilizadas em diferentes empresas de desenvolvimento de software, enquanto as entrevistas possibilitaram a obtenção de informações detalhadas sobre a utilização de ambientes de Integração Contínua para testes de requisitos não funcionais. Entretanto, como a pesquisa foi realizada apenas em empresas dos países nórdicos, a generalização dos resultados para outros contextos deve ser feita com cuidado.

10.3. Avaliação dos resultados

Os resultados apresentados são consistentes com os objetivos da pesquisa e demonstram como os ambientes de Integração Contínua podem apoiar a automação dos testes de requisitos não funcionais. A identificação de ferramentas, métricas e componentes de CI, aliada à proposição do modelo teórico, torna as conclusões bem fundamentadas.

10.4. Aplicação da técnica

A técnica proposta pode ser aplicada em organizações que já utilizam ambientes de Integração Contínua e desejam aprimorar o monitoramento de requisitos não funcionais. A utilização do modelo pode contribuir para a identificação precoce de problemas e para a melhoria contínua da qualidade do software, desde que a empresa possua uma infraestrutura de CI adequada.

10.5. Possíveis melhorias

Como sugestão de aprimoramento, seria interessante que futuras pesquisas realizassem estudos experimentais para validar o modelo em diferentes tipos de projetos e organizações. Além disso, a inclusão de empresas de outros países e setores poderia ampliar a aplicabilidade dos resultados e fornecer evidências adicionais sobre a eficácia da abordagem proposta.

11. Conclusão

11.1. Principais aprendizados

O estudo demonstra que os ambientes de Integração Contínua (CI) podem desempenhar um papel mais amplo do que apenas integrar código e executar testes funcionais. Os dados gerados pelos componentes desses ambientes podem ser utilizados para criar métricas que auxiliam na automação e no monitoramento contínuo dos Requisitos Não Funcionais (NFRs).

11.2. Principal contribuição

A principal contribuição do artigo é a proposição de um modelo teórico-empírico que relaciona os componentes dos ambientes de Integração Contínua aos dados produzidos durante o desenvolvimento, demonstrando como essas informações podem ser transformadas em métricas para a avaliação contínua de requisitos não funcionais.

11.3. Recomendações

Com base nos resultados apresentados, considera-se que a abordagem é promissora para organizações que utilizam Integração Contínua e desejam automatizar a avaliação de requisitos não funcionais. Apesar das limitações e desafios identificados no estudo, a técnica demonstra potencial para melhorar o monitoramento da qualidade do software e favorecer a identificação precoce de problemas.

11.4. Pesquisas futuras

Como trabalhos futuros, os autores sugerem a realização de novos estudos de caso em diferentes contextos e domínios, o aprofundamento da investigação sobre o papel dos componentes da Integração Contínua e a aplicação de algoritmos de aprendizado de máquina para analisar os dados produzidos pelos testes de NFR.

12. Demonstração Prática

Como forma de ilustrar, na prática, o conceito central discutido no artigo analisado, o grupo desenvolveu uma pequena demonstração que reproduz, de maneira simplificada, um dos exemplos apresentados pelos autores (*Example 1*, Seção 4.2 do artigo): a medição automatizada de uma métrica de desempenho – o Tempo Médio de Resposta (*Mean Response Time – MRT*) – integrada a um *pipeline* de Integração Contínua.

12.1. Estrutura da demonstração

A demonstração é composta por três elementos:

- uma API mínima, desenvolvida em Node.js/Express, contendo uma rota simples que simula uma consulta de dados;
- um teste automatizado de desempenho, responsável por realizar múltiplas requisições a essa rota e calcular o MRT, utilizando a mesma fórmula apresentada na Tabela 6 do artigo ($MRT = (A_1 + A_2 + \dots + A_n) / n$);
- um *workflow* do GitHub Actions, que executa esse teste automaticamente a cada envio de código (*push*) ao repositório, reproduzindo o papel do “servidor de CI” descrito no modelo teórico do artigo (Figura 1).

12.2. Cenário de sucesso

No primeiro cenário, a aplicação responde dentro do limite de tempo previamente estabelecido (100 ms). O *pipeline* de CI executa o teste automaticamente, calcula o MRT com base nas requisições coletadas e conclui a execução com sucesso, sinalizando que o requisito não funcional de desempenho foi atendido. A Figura 4 apresenta a execução do *workflow* nesse cenário.

```
✓ Executar teste automatizado de NFR (Performance)

1 ▶ Run npm run test:performance
7
8 > ci-nfr-demo@1.0.0 test:performance
9 > node test/performance-test.js
10
11 == Teste automatizado de NFR: Performance ==
12 Endpoint sob teste: http://localhost:3000/produtos
13 Número de requisições (n): 20
14 Limite aceitável de MRT: 100 ms
15
16 [CI] Subindo a aplicação (source code -> execução)...
17 Servidor rodando em http://localhost:3000
18 [CI] Servidor no ar. Iniciando coleta de dados de performance...
19
20 Requisição 1/20: 33 ms
21 Requisição 2/20: 32 ms
22 Requisição 3/20: 32 ms
23 Requisição 4/20: 32 ms
24 Requisição 5/20: 31 ms
25 Requisição 6/20: 33 ms
26 Requisição 7/20: 32 ms
27 Requisição 8/20: 32 ms
28 Requisição 9/20: 32 ms
29 Requisição 10/20: 32 ms
30 Requisição 11/20: 32 ms
31 Requisição 12/20: 32 ms
32 Requisição 13/20: 33 ms
33 Requisição 14/20: 31 ms
34 Requisição 15/20: 31 ms
35 Requisição 16/20: 32 ms
36 Requisição 17/20: 31 ms
37 Requisição 18/20: 32 ms
38 Requisição 19/20: 31 ms
39 Requisição 20/20: 31 ms
40
41 == Resultado da métrica ==
42 MRT = (33 + 32 + 32 + 32 + 31 + 33 + 32 + 32 + 32 + 32 + 32 + 32 + 33 + 31 + 31 + 32 + 31 + 32 + 31 + 31) / 20
43 MRT = 31.85 ms
44
45 [CI] ✓ OK: MRT (31.85 ms) dentro do limite de 100 ms.
```

Figura 4. Execução do pipeline de CI com o teste de performance aprovado (MRT dentro do limite estabelecido).

12.3. Cenário de falha

No segundo cenário, uma latência artificial foi introduzida propositalmente no código da API, fazendo com que o MRT calculado superasse o limite de 100 ms definido no *workflow*. Como consequência, o *pipeline* de CI falha automaticamente, sinalizando à equipe de desenvolvimento que o requisito não funcional de desempenho não foi atendido – exatamente o comportamento de notificação automática discutido pelos autores como um dos principais benefícios de se utilizar ambientes de CI para testes de NFRs. A Figura 5 apresenta a execução do *workflow* nesse cenário.

```
Executar teste automatizado de NFR (Performance)

1 ▶ Run npm run test:performance
7
8 > ci-nfr-demo@1.0.0 test:performance
9 > node test/performance-test.js
10
11 == Teste automatizado de NFR: Performance ==
12 Endpoint sob teste: http://localhost:3000/produtos
13 Número de requisições (n): 20
14 Limite aceitável de MRT: 100 ms
15
16 [CI] Subindo a aplicação (source code -> execução)...
17 Servidor rodando em http://localhost:3000
18 [CI] Servidor no ar. Iniciando coleta de dados de performance...
19
20 Requisição 1/20: 153 ms
21 Requisição 2/20: 152 ms
22 Requisição 3/20: 151 ms
23 Requisição 4/20: 153 ms
24 Requisição 5/20: 152 ms
25 Requisição 6/20: 152 ms
26 Requisição 7/20: 152 ms
27 Requisição 8/20: 151 ms
28 Requisição 9/20: 152 ms
29 Requisição 10/20: 152 ms
30 Requisição 11/20: 152 ms
31 Requisição 12/20: 151 ms
32 Requisição 13/20: 152 ms
33 Requisição 14/20: 152 ms
34 Requisição 15/20: 152 ms
35 Requisição 16/20: 152 ms
36 Requisição 17/20: 152 ms
37 Requisição 18/20: 152 ms
38 Requisição 19/20: 151 ms
39 Requisição 20/20: 151 ms
40
41 == Resultado da métrica ==
42 MRT = (153 + 152 + 151 + 153 + 152 + 152 + 152 + 151 + 152 + 152 + 152 + 152 + 152 + 152 + 152 + 152 + 151 + 151) / 20
43 MRT = 151.85 ms
44
45 [CI] ✖ FALHA: MRT (151.85 ms) excedeu o limite de 100 ms.
46 [CI] O pipeline será marcado como falho para notificar a equipe.
47 Error: Process completed with exit code 1.
```

Figura 5. Execução do pipeline de CI com o teste de performance reprovado (MRT acima do limite estabelecido).

12.4. Repositório da demonstração

O código-fonte completo da demonstração, incluindo a API, o script de teste automatizado e o *workflow* do GitHub Actions, está disponível no seguinte repositório:

<https://github.com/Fernanda135/ci-nfr-demo>

Referências

Yu, L., Alégroth, E., Chatzipetrou, P., and Gorschek, T. (2023). Automated nfr testing in continuous integration environments: a multi-case study of nordic companies. *Empirical Software Engineering*, 28(6):144.